Full Stack Java Programming – Practical Exam Study Sheet (Exp 1–11)

Code + short explanation + likely viva answers for every experiment, written for a 1-hour revision.

1-hour plan

Time	Topic
0–10 min	Exp 1–3: Java OOP, overloading, inheritance, exceptions
10–22 min	Exp 4–6: Servlets, JSP + JDBC login, JSP implicit objects
22–30 min	Exp 7–8: HTML/CSS/JS static site, JS animation
30–38 min	Exp 9: React useState
38–54 min	Exp 10–11: Spring Boot monolith, JSON REST API
54–60 min	Read the Rapid-fire viva list at the end

Mistakes found in the lab files (already fixed in the code below)

- **Exp 5:** web.xml said `com.example.LoginServlet`, but the servlet has no package. It must be `LoginServlet`. `error.jsp` links to `login.jsp` but the file is `Login.jsp` (Tomcat on Linux is case-sensitive).
- **Exp 8:** `=>` was typed as `={`; `@keyframesspiral` must be `@keyframes spiral`; `var(--angle)` must be `var(--angle)`; `font-family: "sans-serif"` should not be quoted.
- **Exp 10:** the properties lines were merged into one line (dialect + ddl-auto) and the port was 8080 in text but 9090 in the screenshot. Also springdoc 2.x needs Spring Boot 3; with Boot 2.5.3 use springdoc 1.6.x.
- **Exp 9:** `npm install -g create-react-app` is outdated; use `npx create-react-app my-app`.

Exp 1 – Classes, objects, looping, branching

Idea: A class is a blueprint; an object is an instance made with `new`. A constructor initializes the object. A `for` loop goes over an array of objects; `if-else` decides Even/Odd and Senior/Junior.

```
class Student {
    String name;
    int rollNo;

    Student(String name, int rollNo) {      // constructor
        this.name = name;                   // this = current object
        this.rollNo = rollNo;
    }

    static void displayInfo(Student s) {
        System.out.println("Student Name: " + s.name);
        System.out.println("Roll Number: " + s.rollNo);
        if (s.rollNo % 2 == 0) System.out.println("Roll Number is Even");
// branching
        else System.out.println("Roll Number is Odd");
        if (s.rollNo >= 35) System.out.println("Status: Senior Student");
        else System.out.println("Status: Junior Student");
        System.out.println();
    }
}

class Main {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Aayush Gautam", 38),
            new Student("Kritarth Dubey", 32),
            new Student("Rahul Sharma", 25)
        };
        for (int i = 0; i < students.length; i++)   // looping
            Student.displayInfo(students[i]);
    }
}
```

Output: Aayush – 38, Even, Senior · Kritarth – 32, Even, Junior · Rahul – 25, Odd, Junior.

Viva:

- *Constructor?* Special method, same name as class, no return type, runs automatically on `new`.
- *`new` does what?* Allocates heap memory and calls the constructor.
- *`this`?* Reference to the current object.
- *Why is `displayInfo` static?* Called without creating an object: `Student.displayInfo(...)`.

Exp 2 – Method and constructor overloading

Idea: Same name, different parameter list (number, type or order) = overloading. It is **compile-time polymorphism**. Changing only the return type is NOT overloading.

```
class Calculator {
    Calculator() { System.out.println("Default Constructor Called"); }
    Calculator(int a, int b) {
        System.out.println("Parameterized Constructor Called");
        System.out.println("Values: " + a + " and " + b);
    }
    int add(int a, int b)          { return a + b; }
    double add(double a, double b) { return a + b; }
    String add(String a, String b) { return a + " " + b; }
}

class Main {
    public static void main(String[] args) {
        Calculator calc1 = new Calculator();          // default
        Calculator calc2 = new Calculator(5, 10);     // parameterized
        System.out.println();
        Calculator calc = new Calculator();
        System.out.println("Sum of Integers: " + calc.add(10, 20));
        System.out.println("Sum of Doubles: " + calc.add(10.5, 20.5));
        System.out.println("Concatenation of Strings: " + calc.add("Hello",
"World"));
    }
}
```

Output:

```
Default Constructor Called
Parameterized Constructor Called
Values: 5 and 10

Default Constructor Called
Sum of Integers: 30
Sum of Doubles: 31.0
Concatenation of Strings: Hello World
```

Command-line arguments (values passed at run time into `String[] args`): run `java Main 10 20`, then `int a = Integer.parseInt(args[0]);` gives 10.

Viva: Overloading vs overriding? Overloading = same class, different parameters, compile time. Overriding = subclass redefines parent method, run time. Which overload runs for `add(10,20)`? The `int` version (exact match).

Exp 3 – Inheritance and exception handling

Idea: Inheritance lets a class reuse another class using `extends`. Java does not allow multiple inheritance with classes (diamond problem), but allows it with **interfaces** (`implements A, B`). Each public class goes in its own file named after the class.

```
// SINGLE: Dog extends Animal
class Animal { void eat() { System.out.println("Animal is eating"); } }
class Dog extends Animal { void bark() { System.out.println("Dog is
barking"); } }

// MULTILEVEL: Puppy -> Dog -> Animal
class Puppy extends Dog { void weep() { System.out.println("Puppy is
weeping"); } }

// HIERARCHICAL: Dog and Cat both extend Animal
class Cat extends Animal { void meow() { System.out.println("Cat is
meowing"); } }

// MULTIPLE (via interfaces)
interface Father { void showFather(); }
interface Mother { void showMother(); }
class Child implements Father, Mother {
    public void showFather() { System.out.println("Father's Method"); }
    public void showMother() { System.out.println("Mother's Method"); }
```

```
}

// HYBRID (interfaces): B and C extend A, D implements B and C
interface A { void displayA(); }
interface B extends A { void displayB(); }
interface C extends A { void displayC(); }
class D implements B, C {
    public void displayA() { System.out.println("Method of A"); }
    public void displayB() { System.out.println("Method of B"); }
    public void displayC() { System.out.println("Method of C"); }
}
```

Overriding (runtime polymorphism):

```
class Parent { void show() { System.out.println("Parent"); } }
class Kid extends Parent { @Override void show() { System.out.println("Kid");
} }
// Parent p = new Kid(); p.show();  ->  prints Kid (decided by object type)
```

Exception handling:

```
public class FinallyDemo {
    public static void main(String[] args) {
        try {
            int result = 10 / 2;                  // change 2 to 0 to see the
catch run
            System.out.println("Result = " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception occurred.");
        } finally {
            System.out.println("Finally block always executes.");
        }
    }
}
```

Output: `Result = 5` then `Finally block always executes.` (with `10/0`: `Exception occurred.` then the finally line).

Viva:

- *Why no multiple inheritance with classes?* Ambiguity (diamond problem); interfaces solve it.

- *throw vs throws?* `throw` actually throws an exception object; `throws` declares that a method may throw one.
- *Checked vs unchecked?* Checked (IOException) must be handled at compile time; unchecked (ArithmeticException) are runtime exceptions.
- *Does finally always run?* Yes, except when the JVM exits (`System.exit`).

Exp 4 – GenericServlet and HttpServlet (Tomcat)

Idea: A servlet is a Java class that runs on a server and handles requests. **GenericServlet** is protocol-independent (override `service()`). **HttpServlet** extends it and is HTTP-specific (override `doGet()` / `doPost()`). Tomcat is the servlet container. Lifecycle: `init()` once, `service()` per request, `destroy()` once.

Setup in short: unzip Tomcat in C drive → set JDK Path/classpath → run `bin\startup.bat` → open `http://127.0.0.1:8080/` (127.0.0.1 = localhost, 8080 = port) → create `webapps\Sample\WEB-INF\` with `web.xml` and a `classes` folder.

FirstServlet.java (GenericServlet):

```
import java.io.*;
import javax.servlet.*;

public class FirstServlet extends GenericServlet {
    public void service(ServletRequest req, ServletResponse resp)
            throws ServletException, IOException {
        resp.setContentType("text/html");
        PrintWriter pw = resp.getWriter();
        pw.println("<html><head><title>My first Servlet</title></head>");
        pw.println("<body><h2>Welcome To Servlet World!</h2></body></html>");
        pw.close();
    }
}
```

Compile: `set classpath=C:\Program Files\Java\jdk-22\lib\servlet-api.jar` then `javac FirstServlet.java` . Put `FirstServlet.class` in `Sample\WEB-INF\classes` .

web.xml:

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
  <servlet>
    <servlet-name>FirstServlet</servlet-name>
    <servlet-class>FirstServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>FirstServlet</servlet-name>
    <url-pattern>/FirstServlet</url-pattern>
  </servlet-mapping>
</web-app>
```

Run: `http://127.0.0.1:8080/Sample/FirstServlet` → shows **Welcome To Servlet World!**

HttpServlet version (not in your file, but the aim names it, so know it):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SecondServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        resp.setContentType("text/html");
        PrintWriter pw = resp.getWriter();
        pw.println("<h2>Hello from HttpServlet (doGet)</h2>");
        pw.close();
    }
}
```

Viva:

- *GenericServlet vs HttpServlet?* Generic = protocol-independent, one `service()`; Http = HTTP-only, has `doGet` / `doPost`.
- *GET vs POST?* GET sends data in the URL (visible, limited); POST sends it in the body (hidden, used for forms/passwords).
- *Why servlet-api.jar?* It contains the `javax.servlet` classes needed to compile.
- *What is web.xml?* Deployment descriptor that maps URLs to servlets.

Exp 5 – JSP login with JDBC (MySQL) + Servlet

Flow: `Login.jsp` (form) → POST → `LoginServlet` checks MySQL with JDBC → forward to `welcome.jsp` on success or `error.jsp` on failure.

Needs: Tomcat, MySQL, MySQL Connector/J jar placed in `Sample\WEB-INF\lib`.

1. Database (MySQL command line: `mysql -u root -p`):

```
CREATE DATABASE login_db;
USE login_db;
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL,
  password VARCHAR(50) NOT NULL
);
INSERT INTO users (username, password) VALUES ('admin', 'admin123');
```

2. Login.jsp (in `webapps\Sample`):

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
<head><title>Login Page</title></head>
<body>
  <h2>Login Form</h2>
  <form action="LoginServlet" method="post">
    Username: <input type="text" name="username" required /><br><br>
    Password: <input type="password" name="password" required /><br><br>
    <input type="submit" value="Login" />
  </form>
</body>
</html>
```

3. LoginServlet.java (compile, then put the `.class` in `WEB-INF\classes`):

```
import java.io.IOException;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
```

```
    private static final String DB_URL =
"jdbc:mysql://localhost:3306/login_db";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "your_password";   // your
MySQL password

    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        String uname = request.getParameter("username");
        String pwd = request.getParameter("password");
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
// 1 load driver
            Connection conn = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD); // 2 connect
            PreparedStatement ps = conn.prepareStatement(
                "SELECT * FROM users WHERE username=? AND password=?");
// 3 query
            ps.setString(1, uname);
            ps.setString(2, pwd);
            ResultSet rs = ps.executeQuery();
// 4 run

            if (rs.next()) {
// row found = valid
                request.setAttribute("username", uname);
                request.getRequestDispatcher("welcome.jsp").forward(request,
response);
            } else {
                request.getRequestDispatcher("error.jsp").forward(request,
response);
            }
            rs.close(); ps.close(); conn.close();
// 5 close
        } catch (Exception e) {
            throw new ServletException("Login Failed", e);
        }
    }
}
```

4. welcome.jsp:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
<head><title>Welcome</title></head>
<body>
  <h2>Welcome, <%= request.getAttribute("username") %>!</h2>
  <p>You have successfully logged in.</p>
</body>
</html>
```

5. error.jsp:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<html>
<head><title>Login Failed</title></head>
<body>
  <h2>Login Failed</h2>
  <p>Invalid username or password. Try again.</p>
  <a href="Login.jsp">Back to Login</a>
</body>
</html>
```

web.xml: not needed when `@WebServlet` is used. If you use web.xml instead, remove the annotation and write `<servlet-class>LoginServlet</servlet-class>` (no `com.example.`).

Run: restart Tomcat (`shutdown.bat` , then `startup.bat`) → `http://localhost:8080/Sample/Login.jsp` → admin / admin123 → **Welcome, admin!**

Viva:

- *JDBC steps?* Load driver → get connection → create statement → execute query → process ResultSet → close.
- *Why PreparedStatement?* Prevents SQL injection and is precompiled.
- *forward vs redirect?* Forward happens inside the server (URL stays `/LoginServlet`); redirect makes the browser send a new request.
- *Weakness here?* Passwords stored as plain text; real apps hash them.

Exp 6 – JSP implicit and explicit objects

Idea: JSP = HTML with Java inside. Tomcat converts each JSP into a servlet. **Implicit objects** are created automatically by the container (no declaration needed). **Explicit objects** are ones you create yourself (e.g. `Date now = new Date();`).

Components: Directives (`<%@ page %>`, `include`, `taglib`) · Actions (`<jsp:include>`, `<jsp:forward>`, `<jsp:useBean>`) · Scripting (`<% scriptlet %>`, `<%= expression %>`, `<%! declaration %>`) · Implicit objects.

#	Object	Type
1	request	HttpServletRequest
2	response	HttpServletResponse
3	config	ServletConfig
4	application	ServletContext
5	session	HttpSession
6	pageContext	PageContext
7	page	Object (this servlet)
8	exception	Throwable (only on error pages)
9	out	JspWriter / PrintWriter

First.jsp:

```
<html>
<head><title>First Program</title></head>
<body>
<h1>Output of JSP file</h1>
<%
   out.println("Hello World");
%>
</body>
</html>
```

Run: `http://127.0.0.1:8080/Sample/First.jsp`

dispDate.jsp (implicit `out` + explicit `Date` object):

```
<%@page language="java" isThreadSafe="false" import="java.io.*,java.util.*"
%>
<html>
<body bgcolor="white">
<%
    Date now = new Date();               // explicit object
    out.println(now.toString());         // out = implicit object
%>
</body>
</html>
```

Extra example (my own, to show more implicit objects):

```
<%
    String name = request.getParameter("name");          // request
    session.setAttribute("user", name);                  // session
    Integer count = (Integer) application.getAttribute("count");   //
application
    if (count == null) count = 0;
    application.setAttribute("count", count + 1);
    out.println("Hello " + name + ", page visits: " + (count + 1));
%>
```

Run as `.../pagecount.jsp?name=Ravi`.

Viva:

- *How many implicit objects?* 9.
- *session vs application?* session = one user; application = shared by all users of the web app.
- *JSP vs Servlet?* JSP is easier for presentation (HTML-heavy); servlet is better for logic. JSP is compiled into a servlet.
- `<%` *vs* `<%=` *?* Scriptlet runs code; expression prints a value.
- *Which object is only on error pages?* `exception`.

Exp 7 – Static website (HTML + CSS + JavaScript)

Idea: HTML = structure, CSS = looks, JavaScript = behaviour. "Static" means the page content is fixed on the server (no database/server code); JS only adds interactivity in the browser.

index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>My Static Website</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header>
    <h1>Welcome to My Static Website</h1>
    <nav>
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <section>
      <h2>Hello, Dear Student!</h2>
      <p>This is a sample static website built using HTML, CSS, and
JavaScript.</p>
      <button onclick="showMessage()">Click Me</button>
      <p id="message"></p>
    </section>
  </main>
  <footer><p>© 2025 FullStackJavaSyllusSetter</p></footer>
  <script src="script.js"></script>
</body>
</html>
```

style.css:

```
body { font-family: Arial, sans-serif; margin: 0; padding: 0; background-
color: #f9f9f9; color: #333; }
header { background-color: #007acc; color: white; padding: 20px; text-align:
center; }
nav ul { list-style: none; padding: 0; display: flex; justify-content:
center; margin-top: 10px; }
nav li { margin: 0 15px; }
nav a { color: white; text-decoration: none; font-weight: bold; }
main { padding: 20px; text-align: center; }
button { padding: 10px 20px; font-size: 16px; cursor: pointer; }
footer { background-color: #eee; text-align: center; padding: 15px; margin-
top: 40px; }
```

script.js:

```
function showMessage() {
  const message = document.getElementById("message");
  message.textContent = "Thank you for clicking! Enjoy browsing";
}
```

(The original was missing the closing `}`; it is included here.)

Output: blue header with a nav bar; clicking **Click Me** shows the thank-you text under the button.

Viva: Static vs dynamic site? (Dynamic generates pages using server code/DB, e.g. JSP.) What is the DOM? What does `getElementById` do? How is CSS linked? (`<link rel="stylesheet">`) Where do you put the `<script>` tag and why? (End of body, so HTML loads first.)

Exp 8 – JavaScript animation (spiral text)

Idea: Letters of "The Web Designer" move up and down in a sine wave and shrink/grow using a cosine scale, which looks like a rotating spiral. In Chrome, **CSS** does the animation using `@property --angle` and `sin()` / `cos()`; in Firefox `@property` is not supported, so **JavaScript** with `requestAnimationFrame` does it. Files: `index.html`, `style.css`, `script.js`.

index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Animated text rotation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div id="spiral" class="spiral"></div>
  <div id="spiral2" class="spiral"></div>
  <script src="script.js"></script>
</body>
</html>
```

style.css (corrected):

```
body, html {
  height: 100%; margin: 0;
  display: flex; justify-content: center; align-items: center;
  background: #040509; overflow: hidden; font-size: 62.5%;
}
@property --angle {          /* CSS Houdini: makes --angle animatable */
  syntax: '<angle>'; initial-value: 0deg; inherits: false;
}
.spiral {
  display: flex; align-items: center; gap: 10px;
  position: absolute; color: #e0ecef; font-family: sans-serif;
}
@keyframes spiral {
  0%   { --angle: 0deg; }
  100% { --angle: 360deg; }
}
.character {
  font-size: 2.8rem; color: white; text-transform: uppercase;
  transform: translateY(calc(sin(var(--angle)) * 100px))
             scale(calc(cos(var(--angle)) * 0.5 + 0.5));
  animation: spiral 4s linear infinite;
}
@media (max-width: 490px) { .character { font-size: 2.2rem; } }
```

script.js (corrected):

```
let isFirefox = typeof InstallTrigger !== 'undefined';
const words = "The Web Designer";
let ANGLE = 360;
const ANIMATION_DURATION = 4000;

const animation = () => {                 // used only in Firefox
  ANGLE -= 1;
  document.querySelectorAll(".spiral *").forEach((el, i) => {
    const translateY = Math.sin(ANGLE * (Math.PI / 120)) * 100;
    const scale = Math.cos(ANGLE * (Math.PI / 120)) * 0.5 + 0.5;
    const offset = parseInt(el.dataset.offset);
    const delay = i * (ANIMATION_DURATION / 16) - offset;
    setTimeout(() => {
      el.style.transform = `translateY(${translateY}px) scale(${scale})`;
    }, delay);
  });
  requestAnimationFrame(animation);       // repeat on next frame
};

words.split("").forEach((char, i) => {    // one <div> per letter
  const createElement = (offset) => {
    const div = document.createElement("div");
    div.innerText = char;
    div.classList.add("character");
    div.setAttribute("data-offset", offset);
    div.style.animationDelay = `-${i * (ANIMATION_DURATION / 16) -
offset}ms`;
    return div;
  };
  document.querySelector("#spiral").append(createElement(0));
  document.querySelector("#spiral2").append(
    createElement((isFirefox ? 1 : -1) * (ANIMATION_DURATION / 2)));
});

if (isFirefox) { animation(); }
```

Output: the letters of "THE WEB DESIGNER" float in two crossing sine waves on a dark background, endlessly.

Viva:

- `requestAnimationFrame` ? Asks the browser to run a function before the next repaint (about 60 times/sec), giving smooth animation.

- *Why `Math.sin` and `Math.cos`?* Sine gives the up-down position; cosine gives the size, so the letters look like they move in a circle.
- *`querySelector` vs `getElementById`?* querySelector takes any CSS selector.
- *CSS animation vs JS animation?* CSS is simpler and runs on the browser; JS gives full control.

Exp 9 – React Hooks: colour buttons (`useState`)

Idea: React builds UIs from components. A **hook** lets a function component use features like state. `useState('')` returns `[value, setter]`; calling the setter updates the state and React re-renders the component.

Setup: install Node.js (`node -v` , `npm -v` to check) → `npx create-react-app my-app` → `cd my-app` → `npm start` (opens `http://localhost:3000/`).

src/App.js:

```
import React, { useState } from 'react';

function App() {
  const [selectedColor, setSelectedColor] = useState('');   // state

  const handleColorClick = (color) => {
    setSelectedColor(color);                                  // update state
-> re-render
  };

  return (
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
      <h2>Select a Color</h2>
      <div style={{ marginBottom: '20px' }}>
        <button onClick={() => handleColorClick('Red')}
          style={{ margin: '5px', backgroundColor: 'red', color: 'white',
padding: '10px 20px' }}>Red</button>
        <button onClick={() => handleColorClick('Blue')}
          style={{ margin: '5px', backgroundColor: 'blue', color: 'white',
padding: '10px 20px' }}>Blue</button>
        <button onClick={() => handleColorClick('Green')}
          style={{ margin: '5px', backgroundColor: 'green', color: 'white',
padding: '10px 20px' }}>Green</button>
        <button onClick={() => handleColorClick('Yellow')}
```

```
          style={{ margin: '5px', backgroundColor: 'gold', color: 'black',
padding: '10px 20px' }}>Yellow</button>
      </div>

      {selectedColor && (
        <h3>You have selected <span style={{ color:
selectedColor.toLowerCase() }}>{selectedColor}</span>.</h3>
      )}
    </div>
  );
}

export default App;
```

Output: four coloured buttons; click Green → "You have selected Green." with the word in green.

Viva:

- *What is a hook?* A function (starting with `use`) that lets function components use state/lifecycle.
- *Rules of hooks?* Call only at the top level of a component, and only inside React components or custom hooks.
- *State vs props?* State is data owned and changed by the component; props are read-only inputs from the parent.
- *What is JSX?* HTML-like syntax inside JavaScript.
- *Why `{selectedColor && ...}`?* Conditional rendering: shows the message only when a colour is chosen.
- *Virtual DOM?* React compares a lightweight copy of the DOM and updates only what changed.

Exp 10 – Monolithic application using Spring Boot (Book CRUD + H2)

Idea: A **monolithic** application keeps all layers (controller, service, database access) in one deployable unit. Spring Boot auto-configures everything. REST APIs cover CRUD: POST = create, GET = read, PUT = update, DELETE = delete.

Requirements: JDK, VS Code with *Extension Pack for Java* (Microsoft) and *Spring Boot Extension Pack* (VMware), H2 driver.

Create the project in VS Code: Spring Initializr → Maven → Boot version → Java → group `com.example` → artifact `demo` → jar → Java 17 → dependencies: **Spring Web** (REST), **H2 Database** (in-memory DB), **Spring Data JPA** (DB access), **Lombok** (removes boilerplate getters/setters).

src/main/resources/application.properties (each property on its own line):

```
server.port=8080
spring.datasource.url=jdbc:h2:mem:mydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.h2.console.path=/h2
```

Without a database use:
`spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration`

Swagger UI dependency in pom.xml (shows all REST endpoints):

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>   <!-- for Spring Boot 3.x -->
</dependency>
```

Use this exact artifact and version only on Boot 3.x. For Boot 2.x use `springdoc-openapi-ui` version `1.6.15`. Then open `http://localhost:8080/swagger-ui/index.html`.

DemoApplication.java (the hello test from your file):

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    @GetMapping("/hello")
    public String hello() { return "Hello World!"; }
}
```

Run it, then open `http://localhost:8080/hello` → **Hello World!**

Book CRUD (matches the theory: "Book database, CRUD operations"; this part is my addition, as your file stops at hello):

```
// Book.java   (Boot 3.x uses jakarta.persistence; Boot 2.x uses
javax.persistence)
import jakarta.persistence.*;
import lombok.*;

@Entity
@Data @NoArgsConstructor @AllArgsConstructor     // Lombok writes
getters/setters/constructors
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;
}

// BookRepository.java
import org.springframework.data.jpa.repository.JpaRepository;
public interface BookRepository extends JpaRepository<Book, Long> { }

// BookController.java
```

```
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/books")
public class BookController {
    @Autowired private BookRepository repo;

    @PostMapping                public Book add(@RequestBody Book b)
{ return repo.save(b); }
    @GetMapping                 public List<Book> all()
{ return repo.findAll(); }
    @GetMapping("/{id}")        public Book one(@PathVariable Long id)
{ return repo.findById(id).orElse(null); }
    @PutMapping("/{id}")        public Book update(@PathVariable Long id,
@RequestBody Book b) { b.setId(id); return repo.save(b); }
    @DeleteMapping("/{id}")     public void delete(@PathVariable Long id)
{ repo.deleteById(id); }
}
```

Test in Swagger or with POST body `{"title":"Java","author":"Gosling"}` to `/books`. View the data in the H2 console at `http://localhost:8080/h2` (JDBC URL `jdbc:h2:mem:mydb`, user `sa`).

Viva:

- *Monolithic vs microservices?* Monolith = one deployable app with all modules; microservices = many small independent services that talk over APIs.
- *`@SpringBootApplication`?* Combines `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`.
- *`@RestController`?* `@Controller` + `@ResponseBody`; returns data (not a view).
- *What is H2?* In-memory database; data is lost when the app stops.
- *What is JPA/Hibernate?* JPA is the standard for mapping Java objects to tables; Hibernate is the implementation. `ddl-auto=update` creates or updates tables automatically.
- *What does Lombok do?* Generates getters, setters and constructors at compile time via annotations.
- *What is `pom.xml`?* Maven file listing dependencies and build settings.

Exp 11 – Spring Boot REST API with JSON

Idea: Traditional apps send **HTML + data** from the server. Modern apps send only **JSON data**, and the client (React, mobile) builds the screen. This saves bandwidth, is language-independent and is the standard for REST APIs. Spring converts a returned Java object (POJO) into JSON automatically using Jackson.

	JSON	HTML
Purpose	Data exchange	Presentation
Used by	APIs, mobile apps, React/Vue	Web browsers
Server sends	Only data	Data plus formatting

Project structure: `com.firstapp` (base package from start.spring.io) → `Controller/GreetingRestController.java`, `model/HelloMessage.java`, `WebappApplication.java`.

model/HelloMessage.java:

```
package com.firstapp.model;

public class HelloMessage {
    private String title;
    private String content;
    private long timestamp;

    public HelloMessage(String title, String content) {
        this.title = title;
        this.content = content;
        this.timestamp = System.currentTimeMillis();
    }

    // Getters are essential: Jackson uses them to build the JSON
    public String getTitle()    { return title; }
    public String getContent()  { return content; }
    public long getTimestamp()  { return timestamp; }

    public void setTitle(String title)     { this.title = title; }
    public void setContent(String content) { this.content = content; }
}
```

Controller/GreetingRestController.java:

```
package com.firstapp.Controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import com.firstapp.model.HelloMessage;

@RestController                      // = @Controller + @ResponseBody
public class GreetingRestController {

    @GetMapping("/api/hello")        // endpoint URL
    public HelloMessage getJsonGreeting() {
        return new HelloMessage("Welcome to the API", "This is your first
JSON endpoint!");
        // Spring converts the object to JSON
    }
}
```

WebappApplication.java:

```
package com.firstapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class WebappApplication {
    public static void main(String[] args) {
        SpringApplication.run(WebappApplication.class, args);
    }
}
```

Run: run `WebappApplication.java` in VS Code → open `http://localhost:8080/api/hello`.

Output (raw JSON, not an HTML page):

```
{"title":"Welcome to the API","content":"This is your first JSON
endpoint!","timestamp":1759039650003}
```

(The timestamp is the current time in milliseconds, so yours will differ. The screenshot with "Hello Spring Boot!" at `localhost:8080` is a static HTML page placed in `src/main/resources/static/index.html`.)

Viva:

- *What is JSON?* Lightweight, human-readable key-value text format for data exchange.
- *How does the object become JSON?* Spring's Jackson library calls the getters (message converter).
- *Why are getters needed?* Without them Jackson cannot read the private fields.
- *What is REST?* Architecture where resources are accessed via URLs and HTTP methods (GET, POST, PUT, DELETE).
- *What is AJAX?* Updating part of a page by fetching JSON from the server without a full reload.
- *Default port of Spring Boot?* 8080 (change with `server.port`).

Rapid-fire viva list (read last)

Question	Answer
Class vs object	Blueprint vs instance created with `new`
Overloading vs overriding	Same class, different params (compile time) vs subclass redefines parent method (run time)
Multiple inheritance in Java	Only through interfaces
try / catch / finally	Risky code / handle error / always runs
Servlet lifecycle	`init()` → `service()` → `destroy()`
GET vs POST	URL data, visible vs body data, hidden
JDBC steps	Driver → Connection → PreparedStatement → ResultSet → close
JSP implicit objects	9: request, response, config, application, session, pageContext, page, exception, out
HTML / CSS / JS roles	Structure / style / behaviour

Question	Answer
React hook used	`useState` for state; re-renders when state changes
Monolith vs microservices	One unit vs many small independent services
`@RestController`	Returns data (JSON) directly
Why JSON	Light, readable, language-independent, standard for APIs
Default ports	Tomcat 8080, Spring Boot 8080, React dev server 3000, MySQL 3306